**UNIVERSITY OF PADOVA**

# Embedded Real–Time Control

## Laboratory report




Moamen Mohamed Abouelabbas - 2159249
Kiavash Rabbani Soudabi - 2141822

Academic Year 2024-2025

# Contents

**1 Introduction and Preliminaries** **2**
1.1 Introduction . . . 2
1.2 Hardware Overview . . . 2
1.3 Software Overview . . . 2

**2 Lab 1: Line sensor and keypad** **4**
2.1 Relevant theoretical notions . . . 4
2.2 Exercise 1: Handling Keypad Interrupts . . . 5
2.3 Exercise 2: Decoding Keypad Input . . . 5
2.4 Exercise 3: Line Sensor Data via Polling . . . 7
2.5 Exercise 4: Frequency-Controlled LED (Bonus) . . . 8

**3 Lab 2: Open loop control - Camera Stabilizer** **9**
3.1 Relevant theoretical notions . . . 9
3.2 Exercise 1: Tilt Stabilization Using IMU-Based Control Law . . . 9
3.3 Bonus 1: Gyroscope-Based Smooth Pan Control . . . 11
3.4 Bonus 2: Sensor Fusion for Enhanced Tilt . . . 12

**4 Lab 3: Motor Control** **15**
4.1 Introduction . . . 15
4.2 Problem Analysis . . . 15
4.3 Theoretical Background . . . 15
4.4 Implementation and Code Structure . . . 18
4.5 Results . . . 21

**5 Lab 4: Line Follower** **24**
5.1 Introduction . . . 24
5.2 Theroritical Background . . . 24
5.3 Controller Design . . . 26
5.4 Code structure and Implementation . . . 27
5.5 Results . . . 29

**A Appendix: Lab 3** **31**

# 1 Introduction and Preliminaries

## 1.1 Introduction

This laboratory work focused on applying embedded real-time control principles using the Turtlbot mobile robot. Across four labs, students integrated sensors, actuators, and control algorithms in real-world scenarios. In Lab 1, $I^2C$ communication was used to interface a reflectance sensor array and a keypad with the STM32 microcontroller. Lab 2 involved using IMU measurements to control servo motors for stabilizing a camera, demonstrating closed-loop control based on orientation feedback. In Lab 3, motor control was implemented using encoder feedback to estimate rotation and regulate speed. Finally, Lab 4 combined sensor data processing and control by developing a line-following algorithm based on surface reflectivity. Together, these labs provided hands-on experience in embedded system integration and real-time control implementation.

## 1.2 Hardware Overview

The robot used in this laboratory activity is the TurtleBot, a compact differential-drive mobile robot designed for embedded control and sensor integration experiments. It is equipped with various hardware components that support real-time control, perception, and interaction with the environment. The main components of the Turtlbot used in the lab are:

- **STM32 Microcontroller:** Acts as the main processing unit, handling sensor readings, control algorithms, and actuator commands. It provides real-time performance and efficient peripheral management.
- **Two DC Motors with Integrated Encoders:** These motors drive the robot's wheels and allow precise measurement of wheel rotation through built-in position encoders, enabling accurate odometry and velocity control.
- **Inertial Measurement Unit (IMU):** Provides data on angular velocity and linear acceleration, essential for orientation estimation and motion analysis.
- **Servo Motors for Camera Pan and Tilt:** Two servo motors allow control of the camera's orientation. The servo motors are also used to stabilize the camera.
- **Pololu QTR Reflectance Sensor Array:** Used for line detection, this sensor array enables the Turtlbot to follow lines by sensing changes in surface reflectivity.
- **Keypad:** Provides a user interface for manual input or control commands.
- **LED Indicators:** Used for status indication.

## 1.3 Software Overview

The development and testing of the Turtlbot's control and sensing capabilities were supported by a set of software tools tailored for embedded system development and data analysis. The following tools were used throughout the laboratory activities:

- **STM32CubeIDE:** An integrated development environment used for writing, compiling, and debugging C code on the STM32 microcontroller. It provides a unified workspace for managing projects, accessing peripheral registers, and performing real-time debugging.
- **STM32CubeMX:** A graphical configuration tool used to initialize and configure the microcontroller's peripherals, clock settings, GPIOs, and middleware. CubeMX simplifies the setup of timers, ADCs, communication interfaces (e.g., $I^2C$, UART), and generates initialization code compatible with STM32CubeIDE.
- **MATLAB:** Used for data acquisition, visualization, and post-processing of experimental results. MATLAB facilitated the analysis of sensor readings, control responses, and system behavior, allowing for validation and tuning of the implemented algorithms.

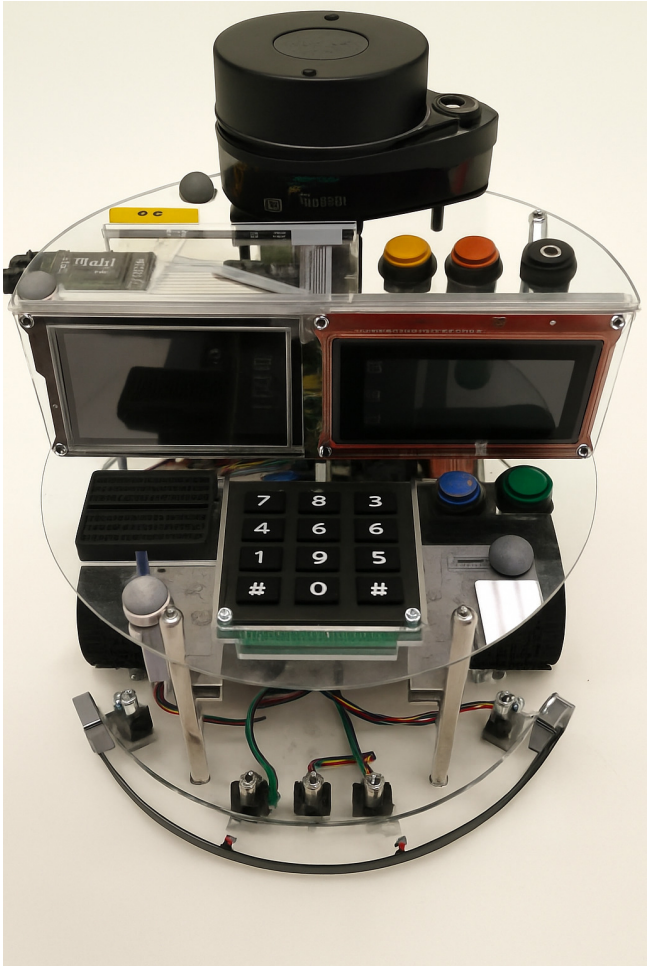



Figure 1: Turtle Bot

# 2 Lab 1: Line sensor and keypad

Purpose of this lab is to learn how to interact with an external device, connected to the micro-controller using serial communication.

## 2.1 Relevant theoretical notions

### 2.1.1 Serial Communication via I2C Protocol

Serial communication is a fundamental method for transmitting data between devices using a single data line, enabling efficient communication with minimal wiring. Among the various serial protocols, I²C (Inter-Integrated Circuit) is widely used for short-distance, low-speed communication between microcontrollers and peripheral devices such as sensors, EEPROMs, and displays. I²C operates on a master-slave architecture using two bidirectional lines: SDA (Serial Data Line) and SCL (Serial Clock Line). This protocol supports multiple devices on the same bus through unique addressing, making it ideal for embedded systems where simplicity and expandability are essential.

### 2.1.2 Hardware Setup

The primary device used is the **sx1509**, a GPIO expander that facilitates easy I/O expansion via a standard 400kHz I2C interface. It is particularly useful in scenarios where the microcontroller has a limited number of I/O pins or where remote serial communication is needed.
The TurtleBot employs two sx1509 devices: one connected to a keypad at I2C address $0x3F$, and the other connected to a line sensor at address $0x3E$. For the keypad, data acquisition is handled through an external interrupt triggered on pin PF4. The scheme of the I2C connection is illusterated in figure (2)

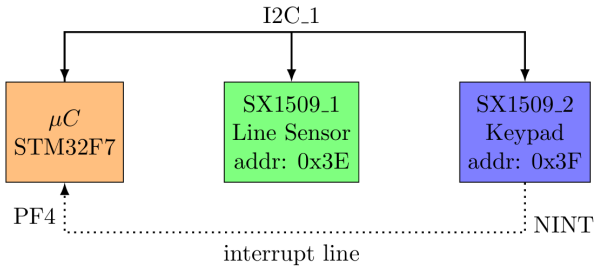



Figure 2: I2C connection scheme

The keypad is configured as a $4 \times 4$ matrix, with each row and column connected to specific GPIO pins of the sx1509 (referred to as sx1509_2). The state of each key is stored in two registers: REG_KEY_DATA_1 (for columns) and REG_KEY_DATA_2 (for rows).

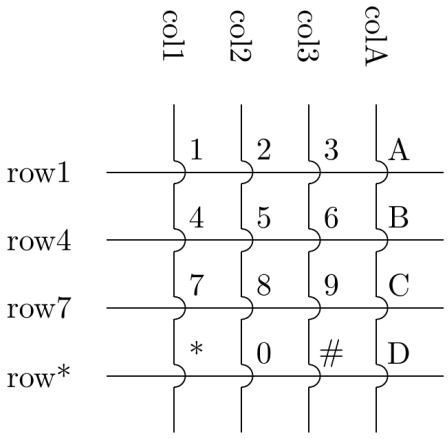



Figure 3: Keypad connection schema

## 2.2 Exercise 1: Handling Keypad Interrupts

Write an ISR that recognizes and correctly handles the keypad interrupts and read the registers REG_KEY_DATA_1 and REG_KEY_DATA_2 inside the ISR.
This exercise is done inside the `HAL_GPIO_EXTI_Callback(uint16_t pin)` which is a callback function provided by the STM32 HAL (Hardware Abstraction Layer) library. It is called automatically whenever a GPIO external interrupt (EXTI) is triggered.

### 2.2.1 Code Contribution

Listing 1: Keypad interrupt handler

```
1         if (GPIO_Pin == GPIO_PIN_4) { // Check if interrupt is from PF4
2             uint8_t key_data_1, key_data_2;
3
4             // Read the keypad column data
5             if (HAL_I2C_Mem_Read(&hi2c1, SX1509_I2C_ADDR2 << 1, REG_KEY_DATA_1,
                  I2C_MEMADD_SIZE_8BIT, &key_data_1, 1, 100) != HAL_OK) {
6                 printf("Error reading REG_KEY_DATA_1\n");
7                 return;
8             }
9
10            // Read the keypad row data
11            if (HAL_I2C_Mem_Read(&hi2c1, SX1509_I2C_ADDR2 << 1, REG_KEY_DATA_2,
                  I2C_MEMADD_SIZE_8BIT, &key_data_2, 1, 100) != HAL_OK) {
12                printf("Error reading REG_KEY_DATA_2\n");
13                return;
14            }
15
16            // Print the detected key status
17            printf("Interrupt triggered on PF4\n");
18            printf("Keypad column data: 0x%02X\n", key_data_1);
19            printf("Keypad row data: 0x%02X\n", key_data_2);
20        }
```

### 2.2.2 Console Output

Listing 2: Output

```
1         Interrupt on pin (16).
2         Keypad column data: 0xFE
3         Keypad row data: 0xF7
4         ...
```

### 2.2.3 Notes

No configuration was required beyond what's prewritten. Interrupts were used as required.

## 2.3 Exercise 2: Decoding Keypad Input

Print which keypad button has been pressed.

### 2.3.1 Code Contribution

The following code reads the keypad column and row data and decode it to the pressed key. Moreover, it returns `"?"` if the data is not valid.

Listing 3: Key decoding routine

```
const char keypad[4][4] = {
    {'1', '2', '3', 'A'},
    {'4', '5', '6', 'B'},
    {'7', '8', '9', 'C'},
    {'*', '0', '#', 'D'}
};

// Exercise2: Function to get key from register values
char decode_key(uint8_t col_data, uint8_t row_data) {
    int row = -1, col = -1;

    // Identify row
    switch (row_data) {
        case 0xFE: row = 3; break;  // Row 1 (bit 0 changed)
        case 0xFD: row = 2; break;  // Row 2 (bit 1 changed)
        case 0xFB: row = 1; break;  // Row 3 (bit 2 changed)
        case 0xF7: row = 0; break;  // Row 4 (bit 3 changed)
    }

    // Identify column
    switch (col_data) {
        case 0xFE: col = 0; break;  // Column 1
        case 0xFD: col = 1; break;  // Column 2
        case 0xFB: col = 2; break;  // Column 3
        case 0xF7: col = 3; break;  // Column 4
    }

    // Return the corresponding key if valid row and column detected
    if (row != -1 && col != -1) {
        return keypad[row][col];
    }

    return '?';  // Unknown key
}

// Decode and print the key
char key = decode_key(key_data_1, key_data_2);
if (key != '?') {
    printf("Keypad column data: 0x%02X\n", key_data_1);
    printf("Keypad row data: 0x%02X\n", key_data_2);
    printf("Key Pressed: %c\n", key);
} else {
    printf("Unknown key press detected\n");
}
```

### 2.3.2 Console Output

Listing 4: Output

```
Interrupt on pin (16).
Keypad column data: 0xFE
Keypad row data: 0xF7
Key Pressed: 1
Interrupt on pin (16).
Keypad column data: 0xFD
Keypad row data: 0xF7
Key Pressed: 2
...
```

### 2.3.3 Notes

In this section, an issue was encountered with the configuration of the row data pins, which were actually set in the opposite order relative to the column data pins. As a result, the output did not correctly reflect the pressed key. After several iterations of trial and error, the correct pin configuration was identified and implemented.

## 2.4 Exercise 3: Line Sensor Data via Polling

Write a routine that reads the status of the line sensor and prints it. The routine must check the status with a polling period of 100ms.

### 2.4.1 Code Contribution

In this part, the line sensor data is read through the I2C bus, and a function is implemented to convert the sensor's original hexadecimal output into a more readable binary format that clearly represents the system's 8-sensor array.

Listing 5: Polling-based line sensor readout

```
void printBinary(uint8_t num) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (num >> i) & 1);  // Extract each bit
        if (i % 4 == 0) printf(" ");  // Add space for readability
    }
    printf("\n");
}

while (1)
{
    /* USER CODE END WHILE */
    HAL_Delay(100);
    uint8_t lineSensorData;

    // Exercise3: Read the status of line sensor
    if (HAL_I2C_Mem_Read(&hi2c1, SX1509_I2C_ADDR1 << 1, REG_DATA_B, 1, &
        lineSensorData, 1, I2C_TIMEOUT) != HAL_OK) {
        printf("Error reading REG_DATA_B\n");
        return 0;
    }

    printf("Line sensor data (BIN): ");
    printBinary(lineSensorData);
}
```

### 2.4.2 Console Output

Listing 6: Output

```
Line sensor data (BIN): 11111111
Line sensor data (BIN): 11101111
...
```

### 2.4.3 Notes

Using binary printouts made interpreting sensor status much easier. Initially unclear outputs were clarified through this strategy.

## 2.5 Exercise 4: Frequency-Controlled LED (Bonus)

Making one of the LED blink with dynamic frequency using the keypad input.

### 2.5.1 Code Contribution

The following code implements the process of receiving a frequency input and converting it into a corresponding delay time for LED blinking.

Listing 7: Input-driven LED toggle

```
// Global variables
char inputBuffer[MAX_DIGITS + 1] = {0};  // Buffer to store user input
int myindex = 0;  // Tracks number of entered digits
int frequency = 0;

// Inside the interrupt handler after decoding the pressed key
if (key >= '0' && key <= '9') {
    if (myindex < MAX_DIGITS) {
        inputBuffer[myindex++] = key;  // Store digit
    }
}
else if (key == '#') {  // End input on '#'
    inputBuffer[myindex] = '\0';  // Null-terminate string
    frequency = atoi(inputBuffer);  // Convert to integer
    printf("\nEntered frequency: %d Hz\n", frequency);

    // Reset buffer for next input
    memset(inputBuffer, 0, sizeof(inputBuffer));
    myindex = 0;
}


// Inside the while loop
if (frequency != 0) {
    uint32_t delayTime = 1000 / (2 * frequency);
    HAL_Delay(delayTime);
    HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_5);
}
```

### 2.5.2 Console Output

Listing 8: Output

```
Entered frequency: 1 Hz
Entered frequency: 2 Hz
Entered frequency: 100 Hz
...
```

### 2.5.3 Notes

Implemented the bonus version with input parsing. LED toggled reliably. This was one of the more technically engaging tasks.

# 3 Lab 2: Open loop control - Camera Stabilizer

The purpose of this lab is to implement an open loop camera stabilizer using the IMU and servo motors.

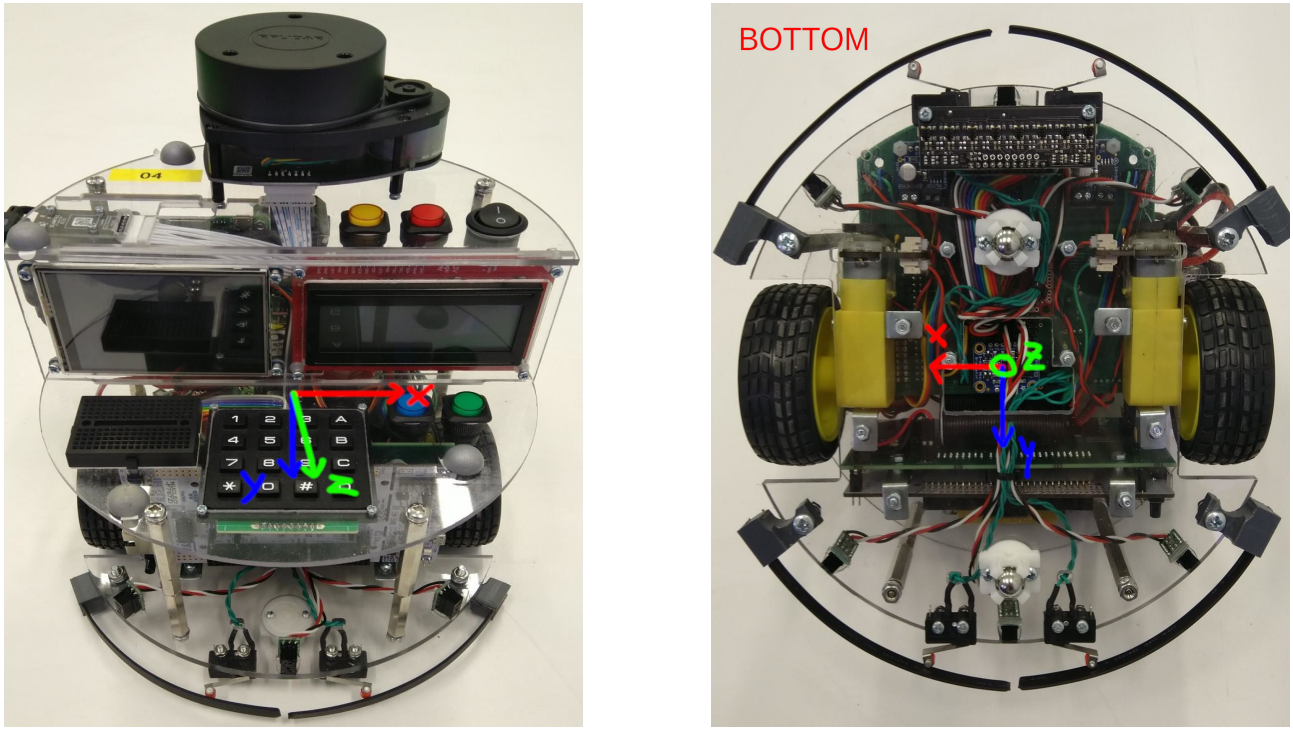
## 3.1 Relevant theoretical notions

In this lab, an IMU is used to get the linear accelerations and angular velocity of the robot. An Inertial Measurement Unit (IMU) is a sensor device that measures and reports an object's specific linear acceleration, angular velocity, and sometimes orientation, using a combination of accelerometers, gyroscopes, and optionally magnetometers. The orientation of the robot can be calculated by fusing measurements from multiple sensor axis.

The IMU on the turtlebot is equipped with a 3-axis accelerometer and a 3-axis gyroscope.

- The accelerometer measures linear acceleration along the x, y, and z axes. It usually provides data in m/s2. It cannot measure pure rotational motion.
- The gyroscope measures angular velocity, indicating the rate of rotation around the x, y, and z axes. It usually provides data rad/s. Unlike an accelerometer, it does not measure linear movement.

The alignments of the sensor axis is reported in Figure (4). Note that the $z - axis$ is pointing downwards (toward the bottom of the robot).




(a) Top View

(b) Bottom View

Figure 4: Axes Alignment.

## 3.2 Exercise 1: Tilt Stabilization Using IMU-Based Control Law

The tilt angle $\theta$ of the TurtleBot, representing rotation around the x-axis, can be estimated using the y-axis output of the onboard accelerometer. Assuming the robot is stationary or moving slowly, the dominant force measured is gravity, allowing orientation to be inferred.
The angle is computed using the following expression:

$$\theta = \arcsin(\frac{a_y}{g}) \tag{1}$$

where $a_y$ is the measured acceleration along the y-axis, and $g = 9.81 m/s^2$ is the gravitational constant. This equation derives from the projection of the gravitational vector onto the sensor's y-axis.

### 3.2.1 Code Contribution

The first step involves acquiring the appropriate measurement from the IMU. The IMU operates at a sampling frequency of 100 Hz, which corresponds to a 20 ms delay within the main execution loop.To access the sensor data, the function `bno055_convert_double_accel_xyz_msq()` is used, which converts the raw accelerometer readings from the BNO055 sensor into double-precision floating-point values representing acceleration.
It is important to note that the output of the inverse sine function used to calculate the tilt angle is in radians; however, the servo control logic requires this value to be expressed in degrees. The control gain is set to -1, ensuring that the camera counteracts the tilt motion of the robot.
Although the `__HAL_TIM_SET_COMPARE` includes an internal call to the `saturate()` function to prevent the PWM signal from exceeding safe limits, applying an additional saturation layer introduces no harm and may provide an extra level of safety.
After logging the relevant data for analysis and debugging, the servo control signal is computed using the following expression:

$$150 + camera_tilt \times (\frac{50.0}{45.0}) \tag{2}$$

This formula linearly maps the camera tilt angle (in degrees) to a corresponding PWM compare value. A tilt of 0° results in a neutral PWM value of 150. A full positive tilt of +45° maps to 200, and a full negative tilt of −45° maps to 100.

Listing 9: Tilt Stabilization

```
1         while (1)
2         {
3             /* USER CODE END WHILE */
4
5             /* USER CODE BEGIN 3 */
6             HAL_Delay(20);
7             bno055_convert_double_accel_xyz_msq(&d_accel_xyz);
8
9             // Read the y acceleration data
10            y_accelerate = d_accel_xyz.y;
11
12            // calculate tilt angle
13            float g = 9.81; // m/s^2
14            tilt = asin(y_accelerate / g) * (180 / M_PI); // convert to degrees
15            camera_tilt = -tilt;
16
17            // limit the tilt to 45 to -45
18            if(camera_tilt > 45) camera_tilt = 45;
19            if(camera_tilt < -45) camera_tilt = -45;
20
21            //update data logger value
22            logger_data.y_accelerate = y_accelerate;
23            logger_data.tilt_angle = (float)tilt;
24            ertc_dlog_update(&logger); // check if someone is connected and
                  waiting for data
25            ertc_dlog_send(&logger, &logger_data, sizeof(logger_data)); // send
                  data
26
27
28            /* update pan-tilt camera */
29            __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2,
30            (uint32_t)saturate((150+camera_tilt*(50.0/45.0)), SERVO_MIN_VALUE,
                  SERVO_MAX_VALUE));
31        }
```

### 3.2.2 Output

The logged data includes the tilt control signal and the Y-axis accelerometer readings. A serial datalogger was used to capture this information during runtime. In the implemented code, the control signal is derived directly from the accelerometer measurements. Specifically, the sensor output is converted into a tilt angle in degrees and then multiplied by –1. The relationship between these signals is illustrated in Figure 5.

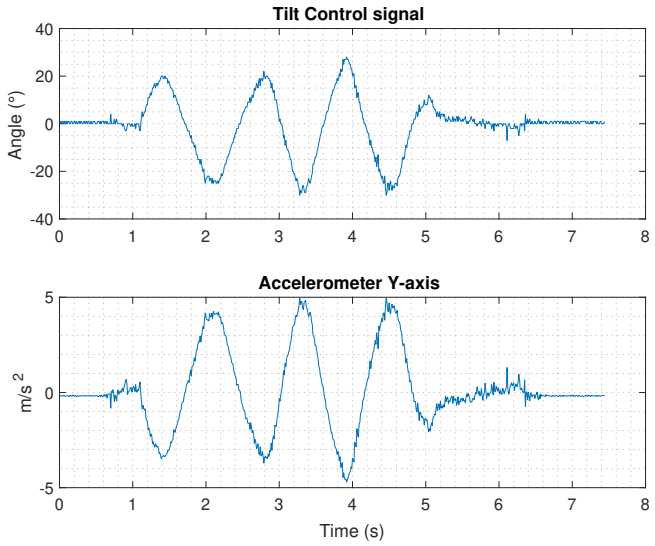



Figure 5: Tilt stabilization output

## 3.3 Bonus 1: Gyroscope-Based Smooth Pan Control

Implement a control algorithm that implements a "smooth pan" control using data from the gyro.

### 3.3.1 Code Contribution

The procedure for acquiring the necessary data follows the same approach used in Exercise 1, utilizing I2C communication to read sensor values. In this case, the gyroscope's Z-axis output is employed to compensate for the robot's horizontal rotation, ensuring that the onboard camera remains oriented forward. A proportional gain $K_p$ is applied to the Z-axis angular velocity to compute the corrective control action. This gain requires tuning based on experimental results. The resulting control signal is then subjected to saturation, similarly to previous implementations, to constrain the commanded pan angle within acceptable physical limits.

Listing 10: Pan control

```
1         while (1)
2           {
3             /* USER CODE END WHILE */
4
5             /* USER CODE BEGIN 3 */
6               HAL_Delay(20);
7               bno055_convert_double_gyro_xyz_rps(&d_gyro_xyz);
8
9               // read the z axis gyroscope value
10              z_gyro = d_gyro_xyz.z;
11
12              // calculate pan control signal using proportional control kp = 30
13              pan = -kp * z_gyro;
14
15              // limit the pan to 45 to -45
16              if(pan > 45) pan = 45;
17              if(pan < -45) pan = -45;
18
```

```
19                //update data logger value
20                logger_data.z_gyro = z_gyro;
21                logger_data.pan_control = (float)pan;
22                ertc_dlog_update(&logger); // check if someone is connected and
                      waiting for data
23                ertc_dlog_send(&logger, &logger_data, sizeof(logger_data)); //
                      send data
24
25
26                /* update pan-tilt camera */
27              __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3,
28                                    (uint32_t)saturate((150+pan*(50.0/45.0)),
                                          SERVO_MIN_VALUE, SERVO_MAX_VALUE));
29            }
```

### 3.3.2 Output

Figure 6 illustrates the system behavior when a proportional gain of $K_p = 30$ is used for pan control.
In the experimental results, pan control was observed to be less responsive compared to tilt stabilization. This can be attributed to the use of a simplified control approach. Specifically, using only a proportional gain, rather than integrating this signal to obtain the angular displacement.

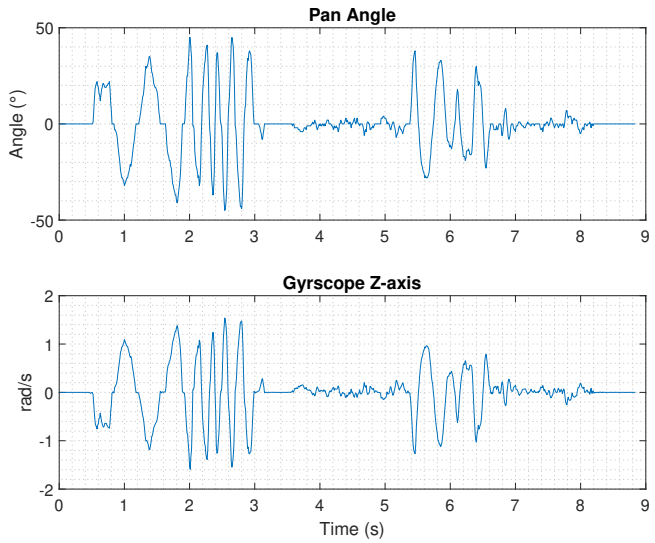



Figure 6: Pan control signal and gyroscope's Z-axis measurements.

## 3.4 Bonus 2: Sensor Fusion for Enhanced Tilt

To improve tilt stabilization, a complementary filter can be applied to fuse accelerometer and gyroscope data, combining the strengths of both sensors. The accelerometer provides reliable long-term orientation data but is susceptible to high-frequency noise and transient disturbances. The gyroscope offers accurate short-term angular velocity measurementsThe complementary filter is defined as: but suffers from drift over time due to integration errors.The complementary filter is defined as:

$$y = \alpha y_1 + (1 - \alpha) y_2 \tag{3}$$

where $\alpha \in [0, 1]$ is a constant that determines the weight of each signal in the final output.

### 3.4.1 Code Contribution

The accelerometer and gyroscope readings are retrieved using library functions. Specifically, the Y-axis acceleration (y_accelerate) and X-axis angular velocity (d_gyro_xyz.x) are used for tilt estimation.
The accelerometer-based tilt angle, denoted $\theta_{accel}$, is computed using the same formulation as previously described for tilt estimation (see Equation 1).
Angular velocity is integrated over time to estimate the tilt angle:

$$\theta_{gyro} = \theta_{fused(prev)} + \omega_x.\Delta t.\frac{\pi}{180} \tag{4}$$

The fused tilt angle is obtained using the complementary filter equation:

$$\theta_{fused} = \alpha.\theta_{gyro} + (1-\alpha).\theta_{accel} \tag{5}$$

The fused angle is converted to a servo control command, constrained between –45° and +45°, and mapped to PWM duty cycle values to actuate the tilt mechanism.

Listing 11: Pan control

```
#define M_PI 3.14159265
#define DT 0.01 // 100 Hz loop
#define ALPHA 0.8

double theta_accel, theta_gyro = 0.0;
static double theta_fused = 0.0;
double y_accelerate;

while (1)
  {
      HAL_Delay(10);
      bno055_convert_double_accel_xyz_msq(&d_accel_xyz);
      bno055_convert_double_gyro_xyz_rps(&d_gyro_xyz);

      // Read the y acceleration data
      y_accelerate = d_accel_xyz.y;

      // calculate tilt angle
      theta_accel = asin(y_accelerate / G) * (180 / M_PI); // convert to
          degrees

     // Integrate gyro angle
    theta_gyro = theta_fused + d_gyro_xyz.x * DT * 180.0 / M_PI;

    // Apply complementary filter
    theta_fused = ALPHA * theta_gyro + (1.0 - ALPHA) * theta_accel;

    // Set tilt value for servo control
    tilt = -theta_fused;

      // limit the tilt to 45 to -45
      if(tilt > 45) tilt = 45;
      if(tilt < -45) tilt = -45;

      //update data logger value
      logger_data.theta_accel = theta_accel;
      logger_data.theta_gyro = theta_gyro;
      logger_data.theta_fused = (double)theta_fused;
      ertc_dlog_update(&logger); // check if someone is connected and
          waiting for data
      ertc_dlog_send(&logger, &logger_data, sizeof(logger_data)); //
          send data

```

```
41
42                 /* update pan-tilt camera */
43                 __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3,
44                                       (uint32_t)saturate((150+tilt*(50.0/45.0)),
                                          SERVO_MIN_VALUE, SERVO_MAX_VALUE));
45             }
```

### 3.4.2 Output

In this implementation, the complementary filter uses a blending factor of $\alpha = 0.8$, which places greater weight on the gyroscope data for estimating the tilt angle. This results in a smoother estimate. However, gyroscope integration introduces gradual drift over time. The accelerometer, although more sensitive to short-term disturbances, provides a stable long-term reference.

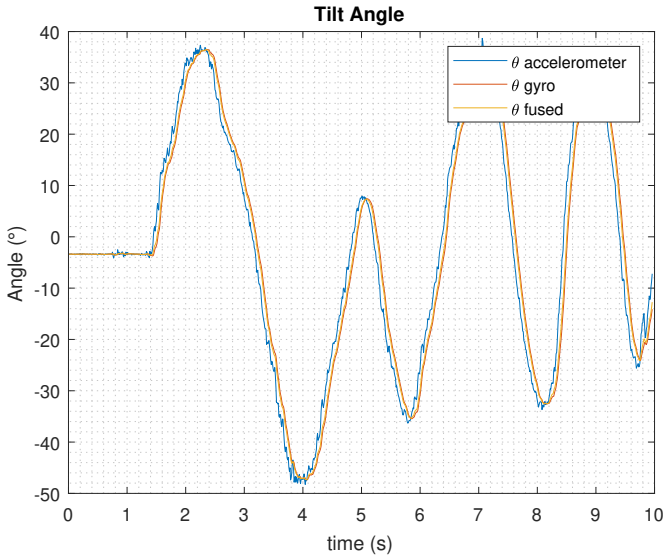



Figure 7: Tilt angle estimates using accelerometer, gyroscope, and complementary filter.

# 4 Lab 3: Motor Control

## 4.1 Introduction

The objective of this lab activity is to implement a PI controller to control the angular speed of the robot wheel, rotating under the action of a DC motor. The problem involves interfacing with the sensor (encoder) and actuator (motor) and implementing the control logic in a closed-loop environment.

## 4.2 Problem Analysis

The problem consists of three main parts:

1. **Interfacing with the encoder:** The encoder generates a pulse when the position changes, so the objective is to compute the current angular speed of the wheel using the encoder measurements.
2. **Calculating the control signal:** Implementing the logic for the PI controller that takes the error of the angular speed and gives a control signal to be executed by the motor.
3. **Interfacing with the motor driver:** This means converting the control signal to a proper PWM signal that can be fed to the motor driver which is responsible of driving the motor.

A schematic for this feedback controller is shown in Figure(8).

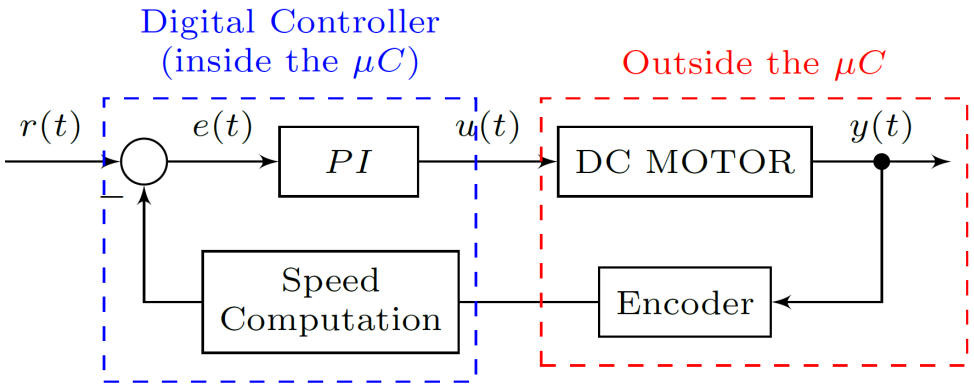



Figure 8: Feedback Control Scheme.

## 4.3 Theoretical Background

### 4.3.1 Motor Model

A DC motor consists of two main components: the stator and the rotor. The stator is the stationary part of the motor and is responsible for generating a magnetic field, either through permanent magnets or electromagnetic windings. The rotor, which is the rotating part, contains windings that produce a magnetic field when an electric current flows through them. A commutator, typically composed of brushes in contact with the rotor windings, acts as a mechanical switch that ensures continuous rotation by periodically reversing the current direction. The interaction between the magnetic field of the stator and that of the rotor generates a repelling force, producing a torque that causes the rotor to spin. This torque is directly proportional to the current through the rotor winding, described by the equation $\tau = k_t i$ where $k_t$ is the torque constant.

The equations of the motor consists of two parts:

- Electrical: From Kirchhoff's Voltage Law (KVL) to the armature circuit

$$\begin{aligned} V_a(t) &= R_a i_a(t) + L_a \frac{di_a(t)}{dt} + V_b(t) \\ &= R_a i_a(t) + L_a \frac{di_a(t)}{dt} + K_b \omega(t) \end{aligned} \tag{6}$$

- Mechanical: From Newton's second law to the rotor

$$J \frac{d\omega(t)}{dt} = T_m(t) - b\omega(t) = K_t i_a(t) - b\omega(t) \tag{7}$$

Using the Laplace transform on (6) and (7) and assuming $K = K_t = K_b$

$$G(s) = \frac{\Omega(s)}{V_a(s)} = \frac{K}{L_a J s^2 + (R_a J + L_a b)s + (R_a b + K^2)} \tag{8}$$

| Symbol | Description | Units |
|---|---|---|
| $V_a(t)$ | Armature voltage (Input voltage) | V |
| $i_a(t)$ | Armature current | A |
| $R_a$ | Armature resistance | Ω |
| $L_a$ | Armature inductance | H |
| $V_b(t)$ | Back EMF (Electromotive Force) | V |
| $K_b$ | Back EMF constant | V/(rad/s) |
| $\omega(t)$ | Angular velocity of the rotor | rad/s |
| $T_m(t)$ | Motor torque developed by the motor | Nm |
| $K_t$ | Torque constant | Nm/A |
| $J$ | Moment of inertia of rotor and load | kg·m$^2$ |
| $b$ | Viscous friction coefficient (damping) | Nm/(rad/s) |

Table 1: DC Motor Variables and Parameters

### 4.3.2 Motor Driver

The motor driver is a chip that can convert the PWM input into a proper output, both in terms of voltage and current, to drive the motor. The chosen chip is DRV8871 by Texas Instruments. is a brushed DC motor driver that can control the motor bidirectionally by implementing a H-bridge. Reported in Figure (9) the pinout of the motor driver DRV8871 and the pins discription in table (2).

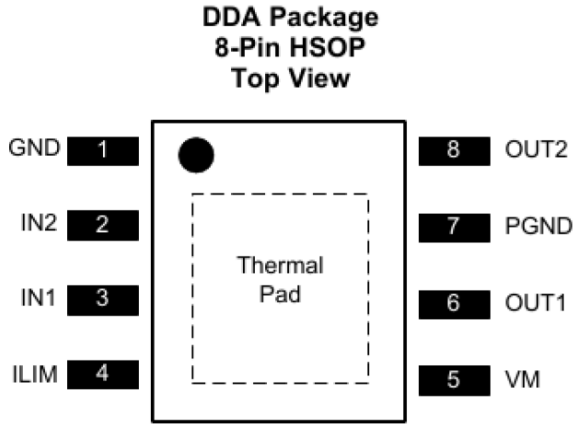



Figure 9: DRV8871 pinout

| Pin name | Pin number | Pin type | Description |
|---|---|---|---|
| IN1 | 3 | Input | Input of the chip that allows controlling the output of the H-bridge. These pins receive the PWM signal from the microcontroller. |
| IN2 | 2 | Input | - |
| OUT1 | 6 | Output | - |
| OUT2 | 8 | Output | H-bridge output that goes to the motor. |

Table 2: Pin configuration table for the H-bridge

**Bridge control**

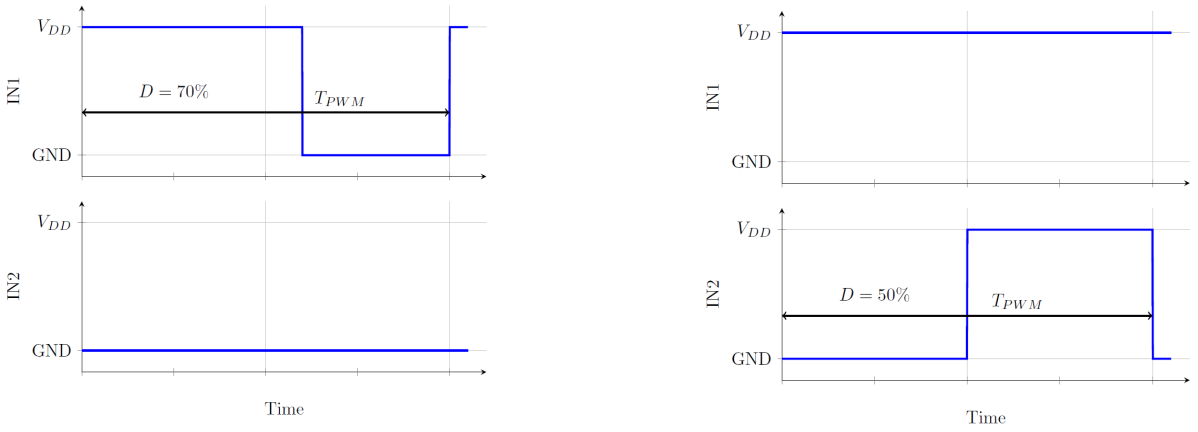
The H-bridge is an electrical circuit used to select the desired driving mode for the motor based on the input of the driver pins. In Table (3), the four configuration of the input pins are reported with the corresponding

| IN1 | IN2 | OUT1 | OUT2 | Mode name |
|---|---|---|---|---|
| 0 | 0 | Hi-z | Hi-z | *Coast* |
| 0 | 1 | L | H | *Reverse* |
| 1 | 0 | H | L | *Forward* |
| 1 | 1 | L | L | *Brake* |

Table 3: Motor driving modes.

mode. The coast mode allows the motor to coast to a stop, which means that the motor is not driven and will move by inertia. On the other hand, the brake mode will stop the motor faster.

When the motor is driven using PWM signal with duty cycle less than 100%, it's possible to alternate between forward and coast modes or forward and brake mode as shown in Figure (10).




(a) Forwrad-Coast mode, speed $\propto$ D

(b) Forward-Break mode, speed $\propto$ 1-D

Figure 10: PWM signal for different driving modes.

### 4.3.3 Position Encoder

The position encoder is a sensor that converts position into an electrical signal. In this context, we focus on a rotary encoder, which is specifically designed to measure angular or rotational positions. More precisely, we use an incremental encoder, which produces a series of pulses corresponding to changes in position, allowing the detection of relative movement rather than absolute position.

The used encoder is of quadrature type which means that the encoder employs two outputs A and B, which are called quadrature outputs, as they are 90 degrees out of phase. The direction of the motor depends on which phase's signal leads over the other. A counter can be used to measure the number of pulses (a pulse can be counted when an edge happens) the number of pulses per round depends on the counting mode. Figure (11) illustrates the different counting modes.The corresponding pulse per round (PPP) is reported in Table (4).

| Mode | ppr motor-side | ppr wheel-side |
|---|---|---|
| 1X | 8 | 960 |
| 2X | 16 | 1920 |
| 4X | 32 | 3840 |

Table 4: Pulses per round for different counting modes with gearbox of ratio of 120:1.

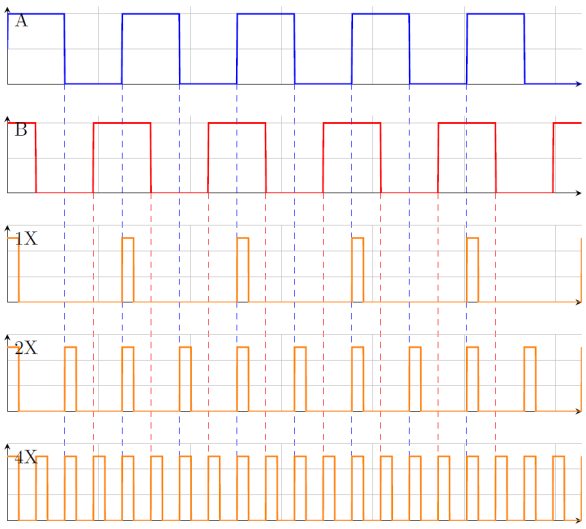



Figure 11: Counting modes illustration.

For example, if the 4X mode is used, the encoder provides 32 pulses for one rotation of the motor which means that for the wheel it's $32 * 120 = 3840$ pulses for one rotation.

## 4.4 Implementation and Code Structure

### 4.4.1 Hardware Setup and Peripheral List

- TIM3 (16bit, general-purpose timer), channel 1 and channel 2: Encoder for motor 1.
- TIM4 (16bit, general-purpose timer), channel 1 and channel 2: Encoder for motor 2.
- TIM6 (16bit, basic timer): Provides the clock for the controller.
- TIM8 (16bit, advanced timer), channel 1 and channel 2: PWM for motor 1.
- TIM8 (16bit, advanced timer), channel 3 and channel 4: PWM for motor 2.

The Figure (12) shows the scheme for hardware connectinos.

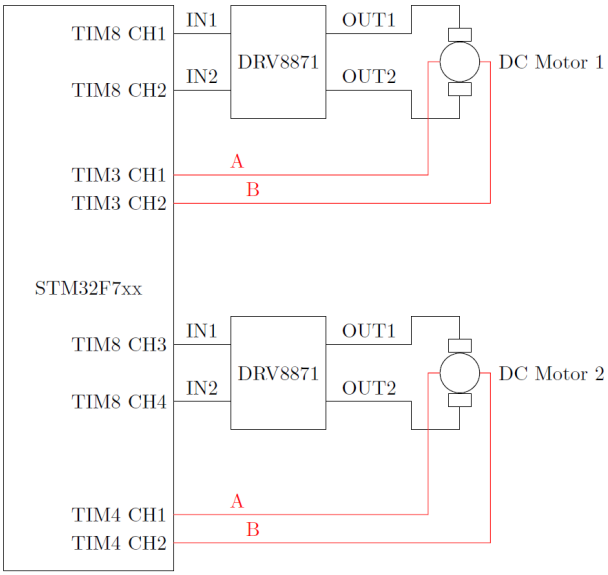



Figure 12: Connection scheme for the timers.

### 4.4.2 Calculating speed from the encoder measurements

The two main focus points in this section is

- Because the timer counter period is set equal to the number of pulses per round, 3840 in case of 4X counting mode, there might be an underflow or overflow in the reading. This is checked and corrected before computing the speed.
- Computing the speed from the encoder measurements.

The logic behind the solution of the first point is to check first whether the timer is counting up or down. For example, if the timer is counting up, overflow happens when the current reading is less than the previous one. To correct the reading in this case, the current reading is added to the interval between the previous reading and the the maximum reading. Similar logic is implemented for the case of underflow. The full implementation is given in code (12).

Listing 12: Getting encoder measurement and correcting over/under flow

```
# define TIM3_ARR_VALUE 3840 // assuming we are using 4X encoder mode
float ReadCurrentSpeed(int motorNumber){
    uint32_t CurrentCount = 0;
    int32_t DiffCount = 0;
    static uint32_t PreviousCount = 0;
    // get diff count for specified motor htim3 for motor 1 and htim4 for motor 2
    switch (motorNumber){
        case 1:
            CurrentCount = __HAL_TIM_GET_COUNTER(&htim3);
            // calculate diff count of motor 1
            if (__HAL_TIM_IS_TIM_COUNTING_DOWN(&htim3))
             {
                /* check for counter underflow */
                if (CurrentCount <= PreviousCount) // Case of no underflow
                    DiffCount = CurrentCount- PreviousCount;
                else // Case of underflow
                    DiffCount =-((TIM3_ARR_VALUE+1)- CurrentCount)- PreviousCount;
             }
             else
             {
                /* check for counter overflow */
                if (CurrentCount >= PreviousCount) // Case of no overflow
                    DiffCount = CurrentCount- PreviousCount;
                else    // Case of overflow
                    DiffCount = ((TIM3_ARR_VALUE+1)- PreviousCount) + CurrentCount;
             }
            PreviousCount = CurrentCount; // Storing the current value
        .
        .
        .
    return currentSpeed;
}
```

As for the second point, the following formula is implemented in code (13)

$$
\begin{aligned}
no.revolutions &= \frac{count\ difference}{PPP(3840)} \\
\omega_{wheel} &= \frac{\text{no. revolutions}}{T_s} * 60\ [rpm]
\end{aligned}
\tag{9}
$$

Listing 13: Wheel speed calculation

```
# define TIM3_ARR_VALUE 3840 // assuming we are using 4X encoder mode
float ReadCurrentSpeed(int motorNumber){
        uint32_t CurrentCount = 0;
        int32_t DiffCount = 0;
        static uint32_t PreviousCount = 0;
        // get diff count for specified motor htim3 for motor 1 and htim4 for motor 2
    .
```

```
8       .
9       .
10          float currentSpeed = 0;
11          // Current speed in rpm
12          currentSpeed = ((float)DiffCount /(float)TIM3_ARR_VALUE)/ TS * 60.0;
13
14      return currentSpeed;
15  }
```

### 4.4.3 Control Logic

To get an initial value for the controller gains, the *MATLAB* toolbox *PID Tuner* is used with the approximated motor transfer function given by

$$P(s) = \frac{\mu}{1+Ts} \cdot e^{-\tau s} \frac{\text{rad/s}}{V} \tag{10}$$

where $\mu = 3.552, \quad T = 0.217, \quad \tau = 0.2$

The implemented controller is a PI controller. The noticeable point in the controller is keeping memory of the value of the integrator which is done by declaring a *static* variable *Iterm* to store the value of the integrator. An Anti-windup is implemented to solve the integrator winding phenomena. The anti-windup, basically, stops the integrating if the output is saturated and the controller output still grows in the wrong direction. see code (14) for the full implementation.

Listing 14: PI Controller with Anti windup

```
1  #define P_gain 0.8
2  #define I_gain 7.5563
3  float calculateControlSignal(float currentSpeed, float referenceSpeed, float* I_term){
4       // PI Controller
5       float error = (referenceSpeed - currentSpeed)*RPM2RADS; //rad/sec
6       float P_term = P_gain*error;
7       float Output_temp = 0;
8
9       *I_term = *I_term + I_gain*TS*error;
10      Output_temp = P_term+*I_term; // volt
11      //float Output_temp = P_term+*I_term; // volt
12      float Output_clamped = clamp(Output_temp);
13
14    // Anti wind-up
15      bool is_saturated = (Output_clamped != Output_temp);
16      bool worsens_saturation = (error * Output_temp > 0);
17      if((is_saturated)&&(worsens_saturation)){
18          *I_term = *I_term - I_gain*TS*error;
19      }
20      return Output_clamped;
21 }
```

### 4.4.4 Motor Control Routine

The ideal sequence of operation is the following:

1. Wait for clock interrupt: the interrupt is raised, in our case by the timer TIM6
2. Read the encoder value and compute the angular speed
3. Compute the error with respect to the reference signal and compute the control signal

Notice that the time required to measure, convert, compute the control signal and provide the proper output from the controller, needs to be less than the sampling period. The control algorithm can be implemented directly inside the timer callback. The motor control routine is shown in code (15).

Listing 15: Motor Control Routine

```
1  /* Speed ctrl routine */
```

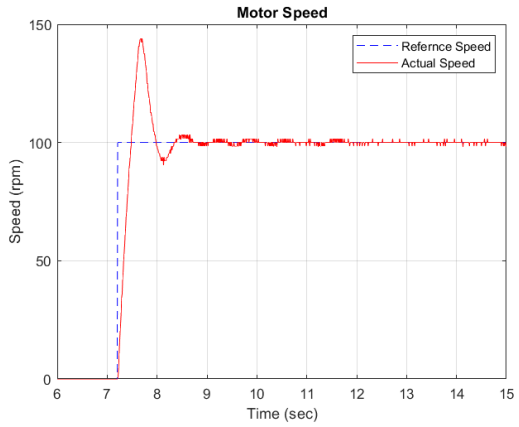
```
2  if(htim->Instance == TIM6)
3  {
4      // Read the speed from the motors
5      float currentSpeed1 = ReadCurrentSpeed(1);
6      float referenceSignal1 = 0;
7      uint32_t ms = HAL_GetTick();
8      float time = ms / 1000.0f;
9
10     if (time < 10.0f){
11         referenceSignal1 = 0.0f;
12     }else if(time > 10.0f){
13         referenceSignal1 = 160.0f;
14     }
15
16     float Output_1 = calculateControlSignal(currentSpeed1, referenceSignal1, &I_term_1);
17     // Set the correct PWM Value
18     int32_t duty1;
19     //calculate duty properly
20     duty1 = Output_1*V2DUTY; //Linear relation between the duty and the voltage
21
22     ForwardBreak(1, duty1);
23         //ForwardCoast(1, duty1);
24 }
```

The functions *ForwardBreak* and *ForwardCoast* is an implementation of the driving modes illustrated in Figure (10). The code is given in Appendix (A).

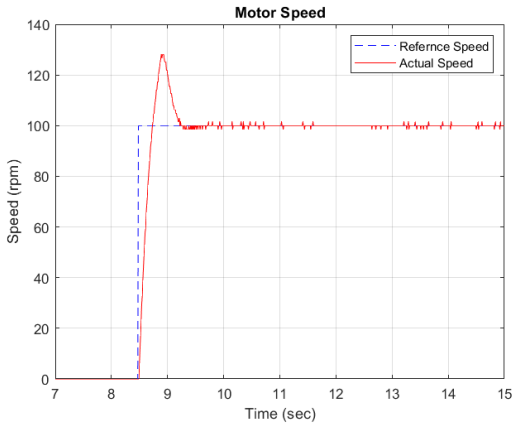
## 4.5 Results

### 4.5.1 Controller Step Response

The controller has been tested with a constant speed reference to evaluate its performance and to compare between using the *Forward Coast* and *Forward Break* modes. The result in Figure (13) shows that the *Forward Break* mode is more precise while the response in *Forward Coast* mode has more oscillations and larger overshoot. For this fact, all the tests will be done using the *Forward Break* mode.




(a) Forward Coast mode




(b) Forward Break mode

Figure 13: Comparison between forward break and forward coast driving mode.

### 4.5.2 Anti-windup Performance

To test the anti-windup, the reference signal is set to exceed the maximum that the motor can output. As shown in Figures (14, 15, 16), the Anti-windup prevents the integrator winding by holding the integrator value when the output saturates.

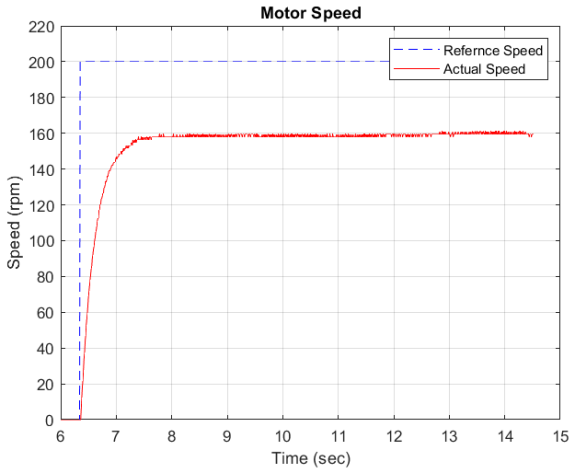
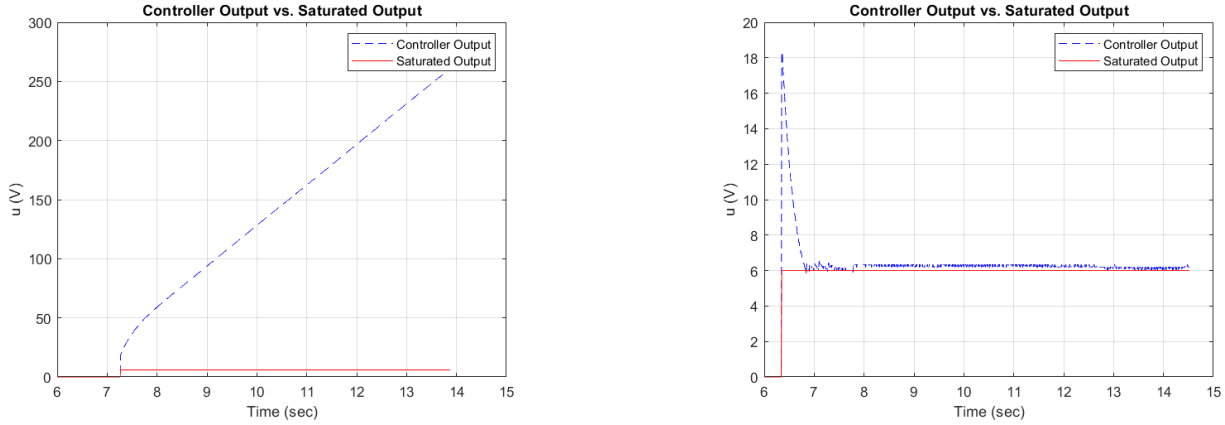


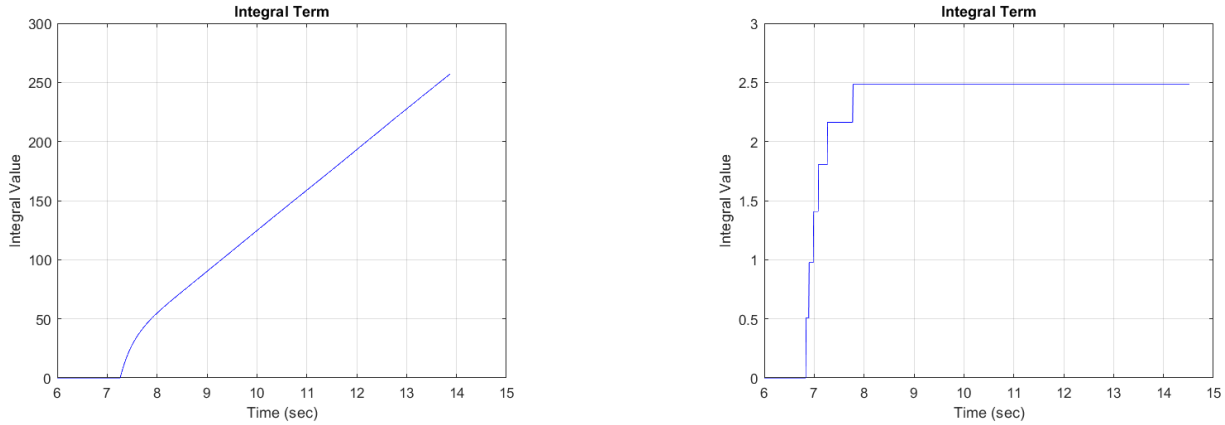

Figure 14: Motor speed in saturation.




(a) No Anti-windup

(b) Using Anti-windup

Figure 15: Effect of using Anti-windup on the controller output.

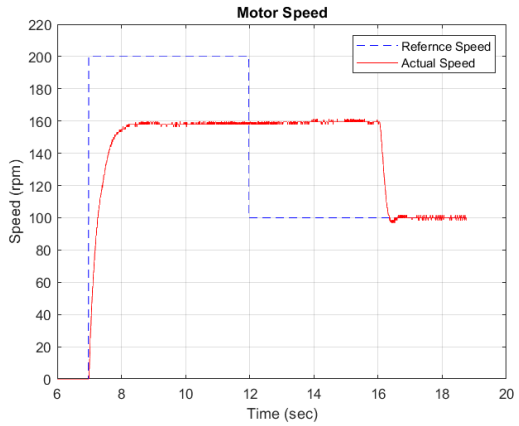



(a) No Anti-windup

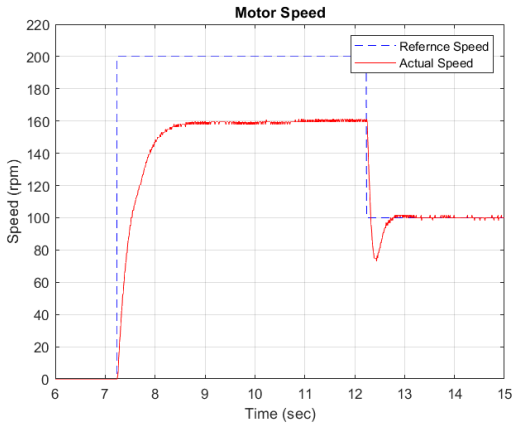
(b) Using Anti-windup

Figure 16: Effect of using Anti-windup on the Integral term.

Another test is done to showcase the effectiveness of using Anti-windup to recover from saturation. The result in Figure (17) shown that without using Anti-windup the motor is still saturated after reducing the reference speed to a smaller value below the motor limit. On the other hand, the Anti-windup is able to recover fast.




(a) No Anti-windup




(b) Using Anti-windup

Figure 17: Effect of using Anti-windup to recover from saturation.

# 5 Lab 4: Line Follower

## 5.1 Introduction

The objective of this lab activity is to design, implement, and test a line-following algorithm. The line is characterized by high contrast compared to the portion of the surface that is not marked as a line. For detecting the line, an array of photo-diodes, each paired with a photo-transistor; each pair forms a digital sensor that allows distinguishing between a dark and a light area.

The experiment focused on processing the sensor data in real time to determine the relative position of the line with respect to the robot's body frame. Based on this input, a control algorithm was developed to adjust the robot's steering by modulating the speed of its wheels to keep it aligned with the line.

## 5.2 Theroritical Background

### 5.2.1 Kinematic model (Differential drive)

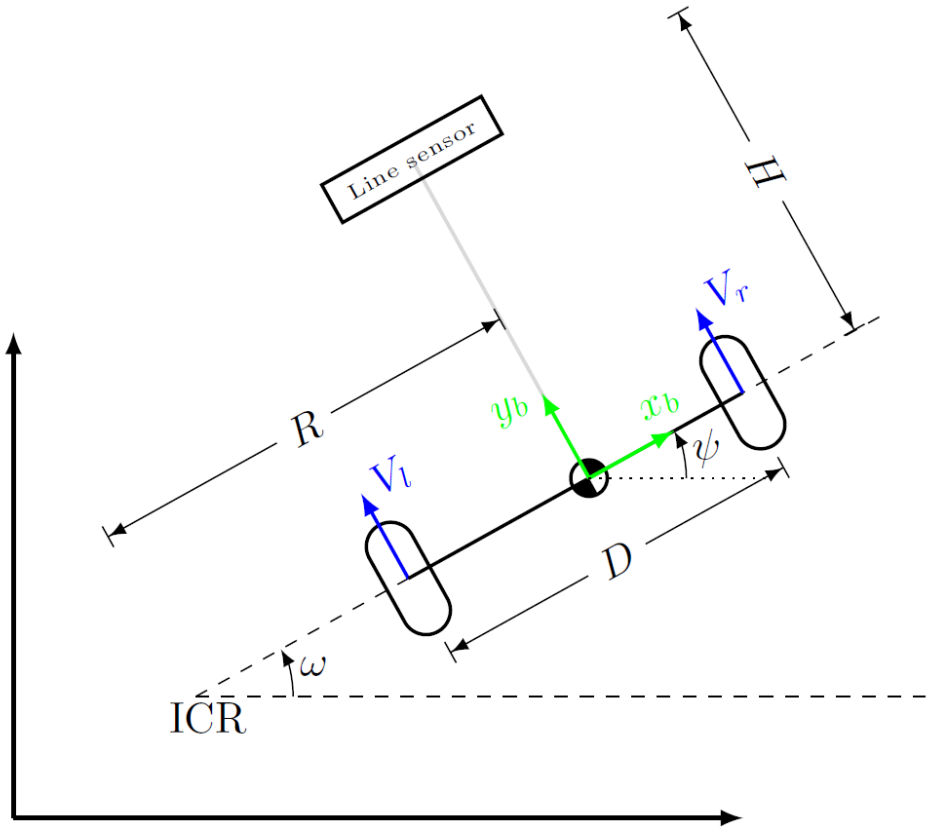



Figure 18: Kinematic model of the robot.

Assumptions:

- Pure roll condition which means that the robot's wheel doesn't slip (Non slipping condition).
- The robot does not move laterally (Non skidding condition).

Based on the pure roll condition, the relationship between the linear velocity of the wheel (ground contact speed) and it's angular speed is defined as:

$$\begin{aligned} V_r &= r \cdot \omega_r \\ V_l &= r \cdot \omega_l \end{aligned} \tag{11}$$

where $\omega_r$ and $\omega_l$ is the angular speed of the right and left wheel, respectively, in $rad/s$ and $r$ is the wheel radius in $m$.

Under the action of $V_r$ and $V_l$ the vehicle rotate by an angular speed $\omega$ around the instantaneous center of rotation (ICR), which is situated at a distance $R$ from the vehicle's center. The following relations hold:

$$\begin{aligned} V_r &= \omega(R + D/2) \\ V_l &= \omega(R - D/2) \end{aligned} \tag{12}$$

By subtracting these two equations, we get:

$$V_r - V_l = \omega D \longrightarrow \omega = \frac{V_r - V_l}{D} \tag{13}$$

where $D$ denotes the distance between the two wheels.

From Figure (18), it's clear that the time derivative of the angle between the global reference axis and the robot reference axis $\dot{\psi}$ is the same as the angular velocity $\omega$. So, by defining $V$ the linear velocity of the vehicle as $V = \omega R$ and by using equation (12), a relation between $V$ and $V_l, V_r$ can be established as

$$\begin{aligned} V_r &= V + \omega\frac{D}{2} &&= V + \dot{\psi}\frac{D}{2} \\ V_l &= V - \omega\frac{D}{2} &&= V - \dot{\psi}\frac{D}{2} \end{aligned} \tag{14}$$

summing the above two equations we get

$$V = \frac{V_r + V_l}{2}$$

### 5.2.2 Error Definition

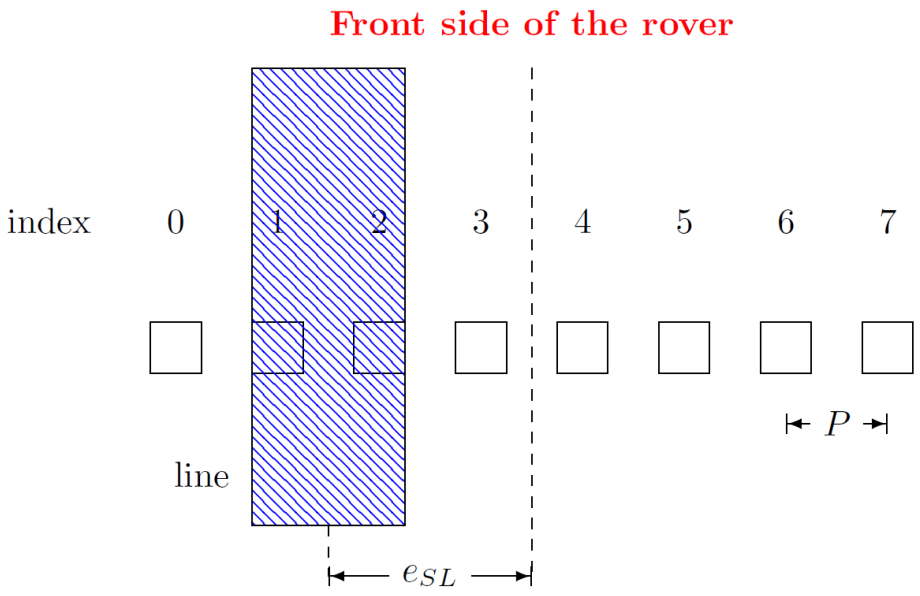



Figure 19: Error definition based on the active sensors in the array.

The logic definition of the line tracking error $e_{SL}$ is given by

$$e_{SL} = \begin{cases} e_{SL} = 0 & \text{when the line is at the center} \\ e_{SL} > 0 & \text{when the line is on the left side of the sensor} \\ e_{SL} < 0 & \text{when the line is on the right side of the sensor} \end{cases} \tag{15}$$

Before defining the mathematical expression for the error, two important parameters

$$b_n = \begin{cases} 1 & \text{if sensor } n \text{ is active} \\ 0 & \text{if sensor } n \text{ is not active} \end{cases} \tag{16}$$

and $\lambda_n$ the distance between the sensor and at position $n$ and the centerline of the sensor array

$$\lambda_n = \left(\frac{N-1}{2} - n\right) P, \quad n \in 0, \ldots, N-1 \tag{17}$$

where $P$ is the pitch distance between sensors, $P = 8\ mm$ and $N$ is the number of sensors in the array, here $N = 8$. Using these parameters, the tracking error $e_{SL}$ is then defined as

$$e_{SL} = \frac{\sum_{n=0}^{N-1} b_n \lambda_n}{\sum_{n=0}^{N-1} b_n} \tag{18}$$

Basically, the error is the weighted average of the distances of the active sensors from the center of the array. The weights are given by the value of the sensor itself, which is either 0 or 1.

Intuitively, the

- Numerator: Adds up the positions of active sensors, each multiplied by its relative distance from the center. So this gives a signed "center of mass" of the active sensors.
- Denominator: Just counts how many sensors are active, to normalize the result (i.e., take an average).

## 5.3 Controller Design

### 5.3.1 Simple Proportional Controller

The objective of this controller is to keep the robot centered at the line by making the robot turn towards the line. The turning speed is proportional to the tracking error $e_{SL}$. This is done by increasing the speed of one wheel and decreasing the speed of the other wheel proportionally to the error. When the error is zero, the two wheels should move at the same speed so the robot move in a straight line. The mathematical expression for this control law is given by equation (19).

$$\begin{aligned} \omega_r &= \omega_0 + K_p \; e_{SL} \\ \omega_l &= \omega_0 - K_p \; e_{SL} \end{aligned} \tag{19}$$

### 5.3.2 Yaw Controller

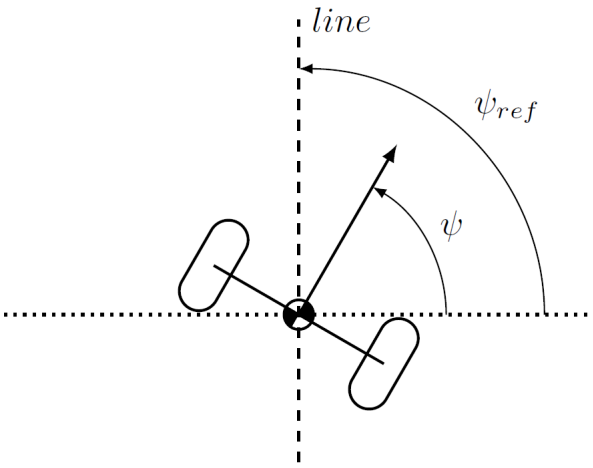



Figure 20: Definition of Yaw angle

According to Figure (20) and the given definition of the tracking error $e_{SL}$, the error in the yaw angle can be defined as

$$\begin{aligned} \psi_{err} &= \psi_{ref} - \psi \\ \psi_{err} &= arctan(\frac{e_{SL}}{H}) \approx \frac{e_{SL}}{H} \end{aligned} \tag{20}$$

where $H$ is the distance between the line sensor and the vehicle's center.

The objective of the controller is to regulate the $\psi_{err}$ to zero. Having each wheel feedback controlled in its speed by PI controllers we can approximate our plant $P(s)$ as an integrator. Under this assumption, it is sufficient, in order to reach our goal, to use a P-controller. The input to the controller is $\psi_{err}$ and the output is $\dot{\psi} = \omega_{robot}$. So using equation (14) and specifying the longitudinal linear velocity of the robot, we can determine the ground contact velocity of each wheel then it can be converted to angular velocity to be the reference for the motor controller.

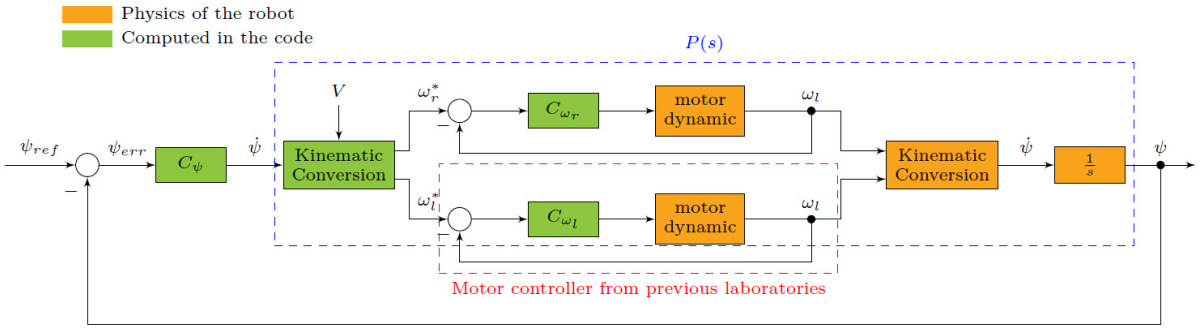



Figure 21: Block diagram showing the yaw controller

## 5.4 Code structure and Implementation

### 5.4.1 Reading Sensor Measurements

First step in the routine is reading the data from the register and store it in an array of zeros and ones.

Listing 16: Reading Sensor Measurements

```
uint8_t LineSensorBinary[8];
void StoreBinary(uint8_t num) {
  for (int i = 7; i >= 0; i--) {
      printf("%d", (num >> i) & 1);  // Extract each bit
      LineSensorBinary[i]= (num >> i) & 1;
      if (i % 4 == 0) printf(" ");  // Add space for readability
  }
  printf("\n");
}

if(htim->Instance == TIM6)
{
    uint8_t lineSensorData;
    if (HAL_I2C_Mem_Read(&hi2c1, SX1509_I2C_ADDR1 << 1, REG_DATA_B, 1, &lineSensorData,
            1, I2C_TIMEOUT) != HAL_OK) {
      printf("Error reading REG_DATA_B\n");
      return 0;
    }
    StoreBinary(lineSensorData);
    .
    .
}
```

### 5.4.2 Calculating the Error

Because of sensor noise, some sensors output value of 1 without being really active. To avoid this noise, the sensor is considered active only if one of the adjacent sensors is also active. This means that two adjacent sensors should detect the black line to consider either of them in the error calculation. Note that the line is wide enough to activate two adjacent sensors. The implementation of this logic is shown in code (17).

Listing 17: Remove the impact of sensor noise

```
float CalculateEsl(){
// Remove the impact of noise on the sensors
    for (int i = 0; i <= 7; i++) {
        if(LineSensorBinary[i] == 1 && i != 0 && i != 7){
            if(LineSensorBinary[i-1] != 1 && LineSensorBinary[i+1] != 1)
                LineSensorBinary[i] = 0;
        }
        else if(i == 0){
```

```
            if(LineSensorBinary[i+1] != 1)
                LineSensorBinary[i] = 0;
        }
        else if(i == 7){
            if(LineSensorBinary[i-1] != 1)
                LineSensorBinary[i] = 0;
        }
    }
.
.
    return (esl);
}
```

The rest of the function is direct implementation of the error definition according to equation (18). A flag *isAllWhite* is raised when all sensors give 0 i.e the robot gets out of track. This flag is used to stop the robot to avoid crashing.

Listing 18: C code using listings

```
float CalculateEsl(){
.
.
    float LineSensorWn[8];
    //calculate Wn the distance between the sensor at position n and the center of
    //the sensor array in meter
    for (int i = 0; i <= 7; i++)
    {
    // pitch is 8mm or 0.008m
    LineSensorWn[i] = (3.5 - i)*0.008;
    }
    float numerator = 0;
    float denominator = 0;
    // Calculate sum of elements of LineSensorWn and LineSensorBinary arrays
    for (int i = 0; i <= 7; i++)
    {
    numerator += (float) LineSensorBinary[i]*LineSensorWn[i];
    denominator += LineSensorBinary[i];
    }
    if (denominator == 0) {
        isAllWhite = true;
        return 0; // Handle the devide by zero exception
    }
    isAllWhite = false;
    float esl = (float) numerator / denominator;
    return (esl);
}
```

### 5.4.3 Yaw Controller

The implementation of the yaw error and the calculation of the angular velocity of each wheel is given in code (19) and (20).

Listing 19: Calculate Yaw Error

```
float CalculateYawError(float esl){
  //Distance between the line sensor and the robot's center H = 0.085 m
    float H = 0.085;
    float yawError = esl/H; //Rad
  return yawError;
}
```

Listing 20: Yaw Controller

```
if(htim->Instance == TIM6)
{
    uint8_t lineSensorData;
    if (HAL_I2C_Mem_Read(&hi2c1, SX1509_I2C_ADDR1 << 1, REG_DATA_B, 1, &lineSensorData, 1,
      printf("Error reading REG_DATA_B\n");
      return 0;
    }
    StoreBinary(lineSensorData);
    //calculate esl
    float Esl = CalculateEsl();

    // Calculate yaw error
    float yawError = CalculateYawError(Esl); // in radian
    float yawKP = 20.0; // tune this based on the results

    float yawRate = yawKP * yawError;

    float nominalSpeed = 120.0 * 2.0 *M_PI * r / 60.0;

    float vr = nominalSpeed + (yawRate * D / 2.0f);
    float vl = nominalSpeed - (yawRate * D / 2.0f);

    float referenceSignal1 = vr * 60.0 / (2.0*M_PI*r); // in rpm
    float referenceSignal2 = vl * 60.0 / (2.0*M_PI*r); // in rpm
    .
    .
}
```

## 5.5 Results

### 5.5.1 Simple Proportional Controller

The controller is tested on a track in the lab. The nominal wheel speed $\omega_0$ is set to 100. The result is shown in Figure (22).

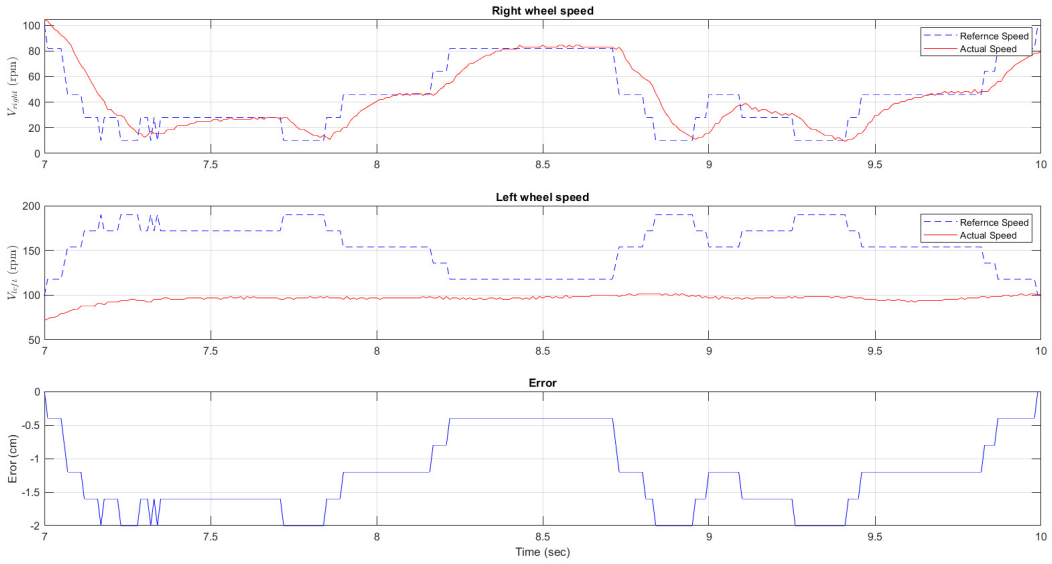



Figure 22: Line following task with simple proportional controller.

The figure shows a small interval of the task to demonstrate the controller effectiveness. Here, the error is negative which means that the line is on the right side of the robot so the robot should turn right. The

commanded behavior is to decrease the speed of the right wheel and increase the speed of the left wheel to turn right. We notice that the actual speed of the left wheel is saturated at 100. A proposed solution is to decrease the nominal speed while turning, but it gives undesirable behavior of excessive turning and getting out of the track.

### 5.5.2 Yaw Controller

The yaw controller is tested under the same conditions as the simple controller. The result in figure (23) shows that the yaw controller is more responsive and makes the response smoother.

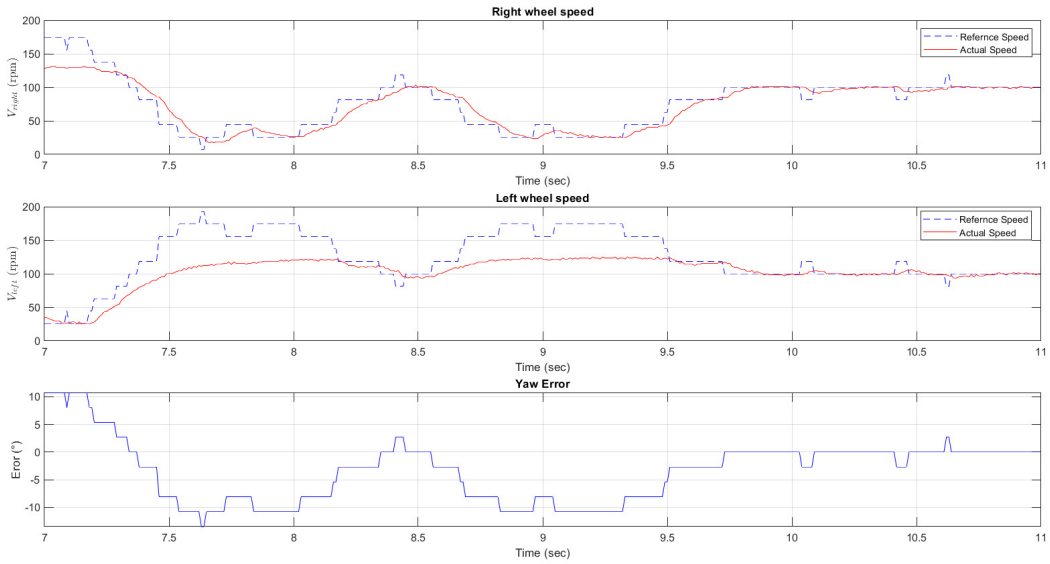



Figure 23: Line following task with yaw controller.

# A Appendix: Lab 3

Listing 21: ForwardCoast and ForwardBreak Implementation

```
void ForwardCoast (int32_t motorNumber, int32_t duty){
switch (motorNumber){
    case 1:
        if (duty >= 0) { // rotate forward
         __HAL_TIM_SET_COMPARE(&htim8, TIM_CHANNEL_1, (uint32_t)duty);
         __HAL_TIM_SET_COMPARE(&htim8, TIM_CHANNEL_2, 0);
         } else { // rotate backward
         __HAL_TIM_SET_COMPARE(&htim8, TIM_CHANNEL_1, 0);
         __HAL_TIM_SET_COMPARE(&htim8, TIM_CHANNEL_2, (uint32_t)-duty);
         }
        break;
    case 2:
        if (duty >= 0) { // rotate forward
         __HAL_TIM_SET_COMPARE(&htim8, TIM_CHANNEL_3, (uint32_t)duty);
         __HAL_TIM_SET_COMPARE(&htim8, TIM_CHANNEL_4, 0);
         } else { // rotate backward
         __HAL_TIM_SET_COMPARE(&htim8, TIM_CHANNEL_3, 0);
         __HAL_TIM_SET_COMPARE(&htim8, TIM_CHANNEL_4, (uint32_t)-duty);
         }
        break;
    }
}
void ForwardBreak (int32_t motorNumber, int32_t duty){
switch (motorNumber){
case 1:
    if (duty >= 0) { // rotate forward
      __HAL_TIM_SET_COMPARE(&htim8, TIM_CHANNEL_1, (uint32_t)TIM8_ARR_VALUE);
      __HAL_TIM_SET_COMPARE(&htim8, TIM_CHANNEL_2, (uint32_t)(TIM8_ARR_VALUE- duty));
     } else { // rotate backward
     __HAL_TIM_SET_COMPARE(&htim8, TIM_CHANNEL_1, (uint32_t)(TIM8_ARR_VALUE - (-duty)));
     __HAL_TIM_SET_COMPARE(&htim8, TIM_CHANNEL_2, (uint32_t)TIM8_ARR_VALUE);
     }
    break;
case 2:
    if (duty >= 0) { // rotate forward
      __HAL_TIM_SET_COMPARE(&htim8, TIM_CHANNEL_3, (uint32_t)TIM8_ARR_VALUE);
      __HAL_TIM_SET_COMPARE(&htim8, TIM_CHANNEL_4, (uint32_t)(TIM8_ARR_VALUE- duty));
     } else { // rotate backward
     __HAL_TIM_SET_COMPARE(&htim8, TIM_CHANNEL_3, (uint32_t)(TIM8_ARR_VALUE - (-duty)));
     __HAL_TIM_SET_COMPARE(&htim8, TIM_CHANNEL_4, (uint32_t)TIM8_ARR_VALUE);
     }
    break;
}
}
```